

客观13-17

题量: 105 满分: 100

作答时间: 05-15 00:19 至 05-20 00:19

智能分析

89.1分

一. 单选题 (共47题, 42.3分)

1. (单选题)不能降低编译期依赖性的是:

A. 自定义类型的数据成员使用指针形式

B. 函数的自定义类型参数使用指针或引用形式

C. 函数返回的自定义类型尽可能使用对象形式

D. 适当使用前置声明和外联实现

我的答案: C

正确答案: C

✓

0.9 分

2. (单选题)体现依赖关系的是:

A. class B { public: void Func(A & pa);};

B. class B { public: A* Func01();};

C. void B::Func02(int n) { A * pa = new A; pa->g(n); delete pa; }

D. 以上三种都是

我的答案: D

正确答案: D

✓

0.9 分

3. (单选题)客观世界中, 最可能具有多对多关联关系的是:

A. 车主和汽车

B. 客户和订单

C. 书籍和作者

D. 中国公民和身份证号

我的答案: C

正确答案: C

✓

0.9 分

答案解析:

4. (单选题)不正确的说法是:

A. 一般关联强调非偶然性的“知道”

B. 聚集关联强调整体-部分关系

C. 聚合关系表示“contain-a”的关系, 整体不负责部分的生存与消亡

D. 客观世界中, 计算机和光驱通常被看作聚合关系

我的答案: D

正确答案: C

✗

0 分

一. 单选题 (42.3分)

1

2

3

6

7

8

11

12

13

16

17

18

21

22

23

26

27

28

31

32

33

36

37

38

41

42

43

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

5. (单选题)下面最能体现组合关系的是:

- A. class B { public: void Func(A* pa); };
- B. class B { A* mpA; };
- C.
- ```
class B {
public: B(A* pa):mpA(pa){ }
private: A * mpA;
};
```
- D.
- ```
class B {
public:
    B():mpA(nullptr){ }
    ~B(){}
    bool CreateA(int n) { mpA=new A(n); }
    bool KillA() { delete mpA; }
private:
    A * mpA;
};
```

我的答案: D 正确答案: D  0.9 分

6. (单选题)客观世界中，具有依赖关系的是:

- A. 母亲和女儿
- B. 学生和研究生
- C. 小猫抓鱼
- D. 人和大脑

我的答案: C 正确答案: C  0.9 分

7. (单选题)

考虑下面代码，说法正确的是:

```
//apple.h
class Apple {
public:
    Apple(int e):power(e) { }
    int getEnergy() const { return power; }
    void eatenBy(Mouse * m) {
        int w = m->getWeight();
        m->setWeight(w+power*0.5);
    }
private:
    int power;
};

//mouse.h
class Mouse {
public:
    Mouse(int w):weight(w) { }
    int getWeight() const { return weight; }
    void setWeight(int w) { weighth =w; }
    void eat(Apple * one) { one->eatenBy(this); }
private:
    int weight;
};
```

作业详情

- B. 类Apple和类Mouse在物理联系上是软关联
- C. 类Apple和类Mouse在逻辑上是双向关联关系
- D. 类Mouse在行为eat的实现中使用了委托

我的答案: D

正确答案: D

✓

0.9 分

答案解析:

8. (单选题)通讯录管理程序中，学生和宿舍的关系应该是：

- A. 依赖
- B. 一般关联
- C. 聚合
- D. 组合

我的答案: D

正确答案: B

✗

0 分

9. (单选题)学生管理程序中，学生和宿舍的关系不应该是：

- A. 依赖
- B. 一般关联
- C. 聚合
- D. 组合

我的答案: A

正确答案: A

✓

0.9 分

10. (单选题)宿舍管理程序中，宿舍和学生的关系不应该是：

- A. 依赖
- B. 一般关联
- C. 聚合
- D. 组合

我的答案: A

正确答案: A

✓

0.9 分

11. (单选题)考虑下面代码，说法最不合理的是：

```
class Dorm {
public:
    Dorm() {
        for(int i=0;i<4;i++)
            mStudents[i] = new Student;
    }
    ~Dorm() {
        for(int i=0;i<4;i++)
            delete mStudents[i];
    }
private:
    Student * mStudents[4];
};
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

作业详情

- B. 类Dorm和类Student是组合关系
- C. 强调了类Dorm和类Student的整体-部分关系
- D. 类Dorm和类Student的实例是共生共死的

我的答案: A 正确答案: A  0.9 分

12. (单选题)属于复用但却不建议使用的是:
- A. 拷贝粘贴修改
 - B. 黑盒复用
 - C. 白盒复用
 - D. 以上都包括

我的答案: A 正确答案: A  0.9 分

13. (单选题)黑盒复用可以通过下面哪种方式实现:
- A. 依赖
 - B. 一般关联
 - C. 聚集关联
 - D. 以上都包括

我的答案: D 正确答案: D  0.9 分

14. (单选题)白盒复用可以通过下面哪种方式实现:
- A. 依赖
 - B. 关联
 - C. 继承
 - D. 以上都包括

我的答案: C 正确答案: C  0.9 分

15. (单选题)已知类A, 属于白盒复用的是:

```
class A {
public:
    void AFunc1( );
    void AFunc2( );
private:
    void AFunc3( );
private:
    int data;
};

A.
class B {
public:
    void BFunc1(A & a) {
        BFunc2( );
        a.AFunc1( );
    }
};
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

作业详情

```
private:
    void BFunc3();
};

B.
class B {
public:
    void BFunc1() {
        BFunc3();
        pA->AFunc1();
    }
    void BFunc2();
private:
    void BFunc3();
private:
    A* pA;
};

C.
class B : public A {
public:
    void BFunc1() {
        BFunc3();
        AFunc1();
    }
    void BFunc2();
private:
    void BFunc3();
};

D. 以上都包括
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

我的答案: C 正确答案: C  0.9 分

16. (单选题) 已知类A、类B，类B中的成员包括：

```
class A {
public:
    void AFunc1();
    void AFunc2();
private:
    void AFunc3();
private:
    int data;
};

class B : public A {
public:
    void BFunc1() {
        BFunc3();
        AFunc1();
    }
    void BFunc2();
private:
    void BFunc3();
private:
    int num;
};
```

A. 缺省的构造、析构、拷贝、赋值函数

B.
B::BFunc1()
B::BFunc2()
B::BFunc3()
B::num

C.

作业详情

A::AFunc3()
A::data

D. 以上都包括

我的答案: D 正确答案: D

✓

0.9 分

17. (单选题)派生类可以从基类继承:

- A. 私有成员
- B. 拷贝构造函数
- C. 析构函数
- D. 友元

我的答案: A 正确答案: A

✓

0.9 分

18. (单选题)关于基类和派生类的描述中错误的是:

- A. 一个派生类可以作为另一个派生类的基类
- B. 派生类至少有一个基类
- C. 派生类只继承了基类的公有成员和保护成员
- D. 派生类的继承方式可以是公有的、保护的或私有的

我的答案: C 正确答案: C

✓

0.9 分

19. (单选题)已知类A和类B, 说法正确的是:

```
class A {  
public:  
    void AFunc1( );  
    void AFunc2( );  
private:  
    void AFunc3( );  
private:  
    int data;  
};  
class B : public A {  
public:  
    void BFunc1( A& a, B & b) {  
        AFunc3( );  
        a.AFunc3();  
        BFunc3();  
        b. BFunc3();  
    }  
private:  
    void BFunc3( );  
private:  
    int num;  
};
```

- A. 由于data在基类A中是私有数据成员, 因此派生类B中的数据成员不包括从基类A中继承来的data
- B. 派生类B中包括从基类A中继承来的方法AFunc3(), 但是由于方法AFunc3()在基类A中是私有的, 所以在BFunc1中直接访问AFunc3()是非法的, 而通过a.AFunc3()访问是合法的
- C. 由于方法BFunc3()是私有的, 私有的成员只有本类或友元可以访问, 因此在BFunc1中直接访问BFunc3()是合法的, 通过b.Bfunc3()访问是非法的

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

我的答案: D 正确答案: D

✓

0.9 分

20. (单选题)派生类的构造函数的成员初始化列表中，不能包含：

- A. 基类的无参构造函数
- B. 基类的拷贝构造函数
- C. 基类中非静态对象成员的初始化
- D. 派生类中非静态对象成员的初始化

我的答案: C 正确答案: C

✓

0.9 分

21. (单选题)如下代码，存在3种情况，说法不正确的是：

```
class base {
public:
    base() { cout << "base::base()" << endl ; }
    base( const base& ) { cout << "base::base( const base&)" << endl ; }
    ~base() { cout << "base::~~base()" << endl ; }
};

class child : public base {
public:
    child() { cout << "child::child()" << endl ; }
    child( const child& ) { cout << "child::child( const child&)" << endl ; }
    ~child() { cout << "child::~~child()" << endl ; }
    int test(int i) { cout << "int child::test(int)" << endl ; return 0 ; }
};

int main() {
    child c1 ;
    cout << "-----" << endl ;
    child c2(c1) ;
    cout << "-----" << endl ;
    return 0;
}
```

- 情况1：把派生类的copy constructor去掉
- 情况2：把基类和派生类的copy constructor都去掉
- 情况3：把基类和派生类的copy constructor都保留

A.

情况1的输出是：

```
base::base()
child::child()
-----
base::base( const base&)
-----
child::~~child()
base::~~base()
child::~~child()
base::~~base()
```

B.

情况2的输出是：

```
base::base()
child::child()
-----
-----
child::~~child()
base::~~base()
child::~~child()
base::~~base()
```

C.

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

作业详情

```
child::child()
-----
base::base()
child::child( const child& )
-----
child::~~child()
base::~~base()
child::~~child()
base::~~base()
```

D. 以上都不对

我的答案: D 正确答案: D  0.9 分

22. (单选题)如下代码，说法不正确的是：

```
class A {
public:
    void F();
    void F(int n);
    void G();
    virtual void H();
private:
    void K();
    int data;
};
class B:public A {
public:
    void BF() {}           // (1)
    void G() {}           // (2)
    void F(int n,int m) {  // (3)
        F();              // (4)
        F(n);             // (5)
    }
    virtual void H() {}    // (6)
};
```

- A. (1)称为newdefine
- B. (2)称为redefine
- C. (3)称为overload，且(4)(5)都是非法的
- D. (6)称为override

我的答案: C 正确答案: C  0.9 分

答案解析:

23. (单选题)public继承方式代表的逻辑关系是：

- A. is a
- B. like a
- C. is a kind of
- D. 以上都包括

我的答案: D 正确答案: D  0.9 分

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

作业详情

- A. has a
- B. contains a
- C. implement of
- D. 以上都包括

我的答案: D 正确答案: D  0.9 分

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

```
25. (单选题)#include <iostream>
using namespace std;
class A {
public:
    A (int n=1):mVal(n)          { cout<<"1"; }
    A (const A& rhs):mVal(rhs.mVal) { cout<<"2";}
    A&operator=(const A& )        { cout<<"3"; return *this;}
    ~A()                          { }
    int mVal;
};
class B:public A {
public:
    B()                          { cout<<"4"; }
    B(const B&)                  { cout<<"5";}
    B&operator=(const B& ) { cout<<"6"; return *this;}
};
int main() {
    B b1;
    B b2(b1);
    b1 = b2;
}
```

上述代码的输出结果是：

- A. 14256
- B. 14156
- C. 142536
- D. 141536

我的答案: B 正确答案: B  0.9 分

26. (单选题)同学们在讨论“优先选择组合而不是继承”时，不禁有疑问：取代继承的为什么是组合？取代继承选择普通关联或聚合不行吗？
针对此疑问的解释，下列正确的是：

- A. 普通关联或聚合通常使用指针，那么就要考虑自定义拷贝、赋值等函数，实现起来麻烦
- B. 使用组合时，可以不使用指针，就不需要考虑自定义拷贝、赋值等函数，实现简单
- C. 继承时子类对象总会拥有父类的全部信息，且负责这些信息的创建和销毁，只有组合具有相似的语义
- D. 使用普通关联或聚合时，父类信息可以是空或非空的，实现其它成员函数时就要随时判断父类信息是否可用，因此程序容易出错

我的答案: C 正确答案: C  0.9 分

答案解析：

作业详情

结合下面给出的代码，能够得出的关于优势和劣势的说法，哪个是错误的？

```
#include <iostream>
using namespace std;
class A {
public:
    void f1(){}
    void f2(){}
};
class InheritA:public A {
public:
    void f3() { f1(); f2(); }
};
class ComboA {
public:
    void f1() { mA.f1(); }
    void f3() { f1(); mA.f2(); }
    A * operator->() { return &mA; }
private:
    A mA;
};
int main() {
    InheritA inherit;
    ComboA combo;
    inherit.f1();
    inherit.f2();
    inherit.f3();
    combo.f1();
    combo.f3();
    combo->f1();
    combo->f2();
}
```

- A. 使用继承，子类能够自动拥有父类原有的公有接口和实现，且代码简单
- B. 使用组合，可以有选择地拥有父类原有的公有接口和实现，但需要额外的程序编码
- C. 使用组合，虽然可以通过重载operator->复用父类的公有接口，但访问这些接口的语法形式会改变，如需将combo.f1 ();改为combo->f1();
- D. 使用组合时，若以对象成员的形式定义组合内容，不用自构造、拷贝、赋值、析构造函数等

我的答案: D

正确答案: D

✓

0.9 分

答案解析:

28. (单选题)有Parent类定义如下：

```
class Parent {
public:
    void f() { }
    void f(int n) { }
};
```

现定义了Parent的子类Child，如下：

```
class Child:public Parent {
public:
    void f(int n,int m) {
        f();
        f(n);
        f(m);
    }
};
```

但Child类不能编译通过，那么下面哪种修改是错误的：

- A. class Child:public Parent {

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

作业详情

```
f();
f(n);
f(m);
}
private:
    using Parent::f;
};

B.
class Child:public Parent {
public:
    void f(int n,int m) {
        Parent::f( );
        Parent::f(n);
        Parent::f(m);
    }
};

C.
class Child:public Parent {
public:
    void f(int n,int m) {
        using Parent::f;
        f( );
        f(n);
        f(m);
    }
};

D.
class Child:public Parent {
public:
    void f( )    { Parent::f( ); }
    void f(int n ) { Parent::f(n); }
    void f(int n,int m) {
        f( );
        f(n);
        f(m);
    }
};
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

我的答案: C 正确答案: C



0.9 分

答案解析:

29. (单选题)已有如下的类定义，其中Derived类以protected方法继承自Base类：

```
class Base {
public:
    void pubBase( );
private:
    int mx;
    int my;
};

class Derived: protected Base {
public:
    void pubDerived( );
protected:
    void protDerived( );
private:
    int mz;
};
```

根据优先使用组合而不是继承的原则，现需要重现定义Derived类。重新定义时，需要注意到程序中已存在一些继承自Derived的类。因此，希望重新定义Derived类后，对这些原来继承自Derived的类产生的影响最小。那么下面哪种是影响最小的？

作业详情

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

```
public:
    Derived( ):mpBase(new Base) {}
    ~Derived( ) { delete mpBase; }
    Derived(const Derived&);
    Derived& operator=(const Derived&);
    void pubDerived( );
protected:
    void pubBase( ) { mpBase->pubBaes( ); }
    void protDerived( );
private:
    Base * mpBase;
    int mz;
};

B.
class Derived {
public:
    Derived( ):mpBase(new Base) {}
    ~Derived() { delete mpBase; }
    Derived(const Derived&);
    Derived& operator=(const Derived&);
    void pubDerived( );
protected:
    void protDerived( );
private:
    Base * mpBase;
    int mz;
};

C.
class Derived {
public:
    void pubDerived( );
protected:
    void pubBase( ) { mBase.pubBaes( ); }
    void protDerived( );
private:
    Base mBase;
    int mz;
};

D.
class Derived {
public:
    void pubDerived( );
protected:
    void protDerived( );
private:
    Base mBase;
    int mz;
};
```

我的答案: C 正确答案: C



0.9 分

答案解析:

30. (单选题)public继承下的向上类型转换一般是指:
- A. 将派生类的对象指针转换成基类的对象指针
 - B. 将派生类的对象引用转换成基类的对象引用
 - C. 将派生类的对象转换成基类的对象
 - D. 以上都包括

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

31. (单选题)

关于下面代码，说法正确的是：

```
class Bike {
public:
    void move( );
    //略
};
class Player : private Bike{    // (1)
public:
    //略
private:
    //略
};
int main( ) {
    Player player;
    player.move( );           // (2)
    Bike * pb = (Bike *)(&player); // (3)
    pb->move( );              // (4)
    Bike & b = (Bike ) (player); // (5)
}
```

- A. (1)处private继承方式不合理
- B. (2)处通过派生类对象访问继承过来的基类公有成员函数合理
- C. (3)处强制类型转换是被禁止的
- D. (4)处通过基类对象指针访问基类公有成员函数合理
- E. (5)处强制类型转换合理

我的答案: D 正确答案: D  0.9 分

答案解析:

32. (单选题)

已知下面代码，则选项中最可能不合理的是：

```
class Car {
public:
    Car(){/*略*/}
    void doFunc( ){/*略*/};
    //其它略
};
class MTCar:public Car {
public:
    MTCar(){/*略*/}
    void newFunc( ){/*略*/}
    //其它略
};
class Foo {
public:
    Foo(Car * pObj=NULLPTR):mp(pObj){}
    void func(Car & obj){
        obj.doFunc( );
    }
private:
    Car * mp;
    //其它略
};
```

A.

MTCar mc;

作业详情

B.
MTCar mc;
Foo fobj;
fobj.func(mc);

C.
MTCar mc;
Car *pc=(Car*)&mc;

D.
Car c;
MTCar* pmc=(MTCar*)&c;

E.
MTCar mc;
Car c;
c=mc;

我的答案: D

正确答案: D

0.9 分

答案解析:

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

33. (单选题)已知下面代码，则选项中最合理的是：

```
class Car {
public:
    Car(){/*略*/}
    void doFunc() { /*略*/ };
    //其它略
};
class MTCar:public Car {
    MTCar(){ /*略*/ }
    //其它略
};
class ATCar:public Car {
    ATCar(){ /*略*/ }
    //其它略
};
class Foo {
public:
    void func(MTCar mobj){ // (1)
        mobj.doFunc();
    }
    void func(ATCar aobj){ // (2)
        aobj.doFunc();
    }
}
private:
    MTCar mc;           // (3)
    ATCar ac;           // (4)
    //其它略
};
```

- A. (1)(2)(3)(4)都是合理的，不需要修改
- B. (1)(2)的形参类型都改为指针型，(3)(4)的数据成员类型都改为引用型
- C. (1)(2)的形参类型都改为引用型，(3)(4)的数据成员类型都改为指针型
- D.
将(1)(2)合并，修改为void func(Car obj){ obj.doFunc();}
将(3)(4)合并，修改为Car c;
- E.
将(1)(2)合并，修改为void func(Car* pobj){ pobj->doFunc();}
将(3)(4)合并，修改为Car& rc;
- F.
将(1)(2)合并，修改为void func(const Car& robj){ robj.doFunc();}
将(3)(4)合并，修改为const Car* pc;

作业详情

将(3)(4)合并，修改为Car* pc;

我的答案: G 正确答案: G  0.9 分

34. (单选题)C++中的类型转换方式有:

- A. 内置类型的自动转换
- B. 构造函数转换
- C. 定义自动转换函数
- D. public继承下的向上类型自动转换
- E. 使用类型转换操作符
- F. 以上都包括

我的答案: F 正确答案: F  0.9 分

35. (单选题)C++中的类型转换操作符有:

- A. static_cast
- B. const_cast
- C. reinterpret_cast
- D. dynamic_cast
- E. 以上都包括

我的答案: E 正确答案: E  0.9 分

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

```
class A {  
  
public:  
  
    A() { /*略*/ }  
  
    //略  
  
};  
  
A * p1 = new A;  
  
const A * const p2 = p1; // (1)  
  
const A * p3 = const_cast<const A *>(p2); // (2)  
  
A * const p4 = const_cast<A * const>(p2); // (3)  
  
A * p5 = const_cast<A *>(p2); // (4)
```

一. 单选题 (42.3分)

- 1
- 2
- 3
- 6
- 7
- 8
- 11
- 12
- 13
- 16
- 17
- 18
- 21
- 22
- 23
- 26
- 27
- 28
- 31
- 32
- 33
- 36
- 37
- 38
- 41
- 42
- 43

36. (单选题)

- A. (1)是合法的
- B. (2)是合法的
- C. (3)是合法的
- D. (4)是合法的
- E. (1)(2)(3)(4)中有不合法的

我的答案: E 正确答案: E  0.9 分

答案解析:

37. (单选题)

动态类型转换操作符的一般格式为dynamic_cast<T>(exp)，其中T只能是：

- A. 类的指针
- B. 类的引用
- C. void *
- D. 以上都包括

我的答案: D 正确答案: D  0.9 分

答案解析:

作业详情

```
Data(int n=0):mx(n),mReadCount(0) { }
int getX() const;
int readCount() const { return mReadCount;}
private:
int mx;
int mReadCount; ///记录getX的调用次数
};
```

若希望Data类中的mReadCount成员，记录本对象的累计getX次数，那么getX()的正确实现是：

- A.
- ```
int Data::getX() const {
 ++mReadCount;
 return mx;
}
```
- B.
- ```
int Data::getX() const {
    Data * p =reinterpret_cast<Data *>( this );
    ++p->mReadCount;
    return mx;
}
```
- C.
- ```
int Data::getX() const {
 Data * p =static_cast<Data *>(this);
 ++p->mReadCount;
 return mx;
}
```
- D.
- ```
int Data::getX() const {
    Data * p =const_cast<Data *>( this );
    ++p->mReadCount;
    return mx;
}
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

我的答案: D 正确答案: D



0.9 分

答案解析：

39. (单选题)

阅读下面的程序：

```
#include <iostream>
using namespace std;
class Fruit {
public:
    virtual ~Fruit() {}
};
class Apple: public Fruit { };
class Melon: public Fruit { };

class Basket{
public:
    void addFruit(Fruit* fruit) {
        if (fruit == nullptr) return;
        for(int i = 0; i < 10; ++i)
            if(mFruits[i] == nullptr) { //1
                mFruits[i] = fruit; //2
                return;
            }
    }
    int appleCount() const {
        int num = 0;
        for(Fruit * ptem : mFruits){
            Apple * p = dynamic_cast<Apple *>(ptem);
            if(p) ++num;
        }
    }
};
```

作业详情

```
    }
private:
    Fruit* mFruits[10] = { };
};

int main( ) {
    Apple apples[2];
    Fruit* fs[6] = { new Apple, new Melon, new Apple, nullptr, new Melon };
    Basket basket;

    basket.addFruit(&apples[0]);
    basket.addFruit(&apples[1]);
    for(Fruit* pItem : fs) {
        basket.addFruit(pItem);
    }

    cout << basket.appleCount( ) << endl;

    for(Fruit* pItem : fs)
        delete pItem;
}
那么下面说法错误的是：
```

- A. 程序能够输出4
- B. 程序执行时，会产生内存释放不完整的问题.
- C. 程序执行过程中，累计向Basket中放入了6个水果
- D. 程序中//1处，保证当mFruits数组元素是空指针时，才执行//2中的语句

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

我的答案: B 正确答案: B  0.9 分

答案解析:

```
40. (单选题)已知下面程序代码，那么对象d的内存布局通常是：
class A {
public:
    void FA( );
private:
    int mA;
};
class B: public A {
public:
    void FB( );
private:
    int mB;
};
class C: public A {
public:
    void FC( );
private:
    int mC;
};
class D: public B, public C {
public:
    void FD( );
private:
    int mD;
};
int main( ) {
    D d;
    return 0;
}
```

作业详情

A.

D::FD

B.

A::mA
A::FA
B::mB
B::FB
C::mC
C::FC
D::mD
D::FD

C.

A::mA
B::mB
C::mC
D::mD

D.

B::A::mA
B::mB
C::A::mA
C::mC
D::mD

E. 以上都不对

我的答案: D 正确答案: D  0.9 分

答案解析:

一. 单选题 (42.3分)

1

2

3

6

7

8

11

12

13

16

17

18

21

22

23

26

27

28

31

32

33

36

37

38

41

42

43

41. (单选题)已知下面程序代码，那么关于该程序说法正确的是：

```
class A {
public:
    A():num(1) {}
    int f() { return 55; }
protected:
    int num;
};
class B{
public:
    B():num(2) {}
    int f() { return 88; }
protected:
    int num;
};
class C: public A, public B {
public:
    // (1)
    void k() {
        cout << f() << endl; // (2)
        cout << num << endl; // (3)
    }
};
```

A. 该程序不存在问题，能够编译通过

B. 该程序存在名字冲突问题，在(1)处增加1条语句using A::num;即可编译通过

C. 该程序存在名字冲突问题，在(2)处将f()修改为B::f()即可编译通过

D. 该程序存在名字冲突问题，在(1)处增加2条语句using A::num; using B::f()即可编译通过

E. 以上都不对

https://mooc1.chaoxing.com/mooc-ans/mooc2/work/view?courseId=201642417&classId=93222363&cpi=352876303&workId=35022770&answ...

19/37

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

42. (单选题)C++引入虚基类的机制是因为:

A. 多重继承下, 可能导致名字冲突

B. 为了实现更加灵活的多态

C. C++不支持接口, 故以虚基类作为补充

D. 为了基类数据提供更好的保护

我的答案: A 正确答案: A

✓

0.9 分

43. (单选题)下列对虚基类声明正确的是:

A.
class virtual B: public A
{ /*略*/ }

B.
class B: virtual public A
{ /*略*/ }

C.
class B: public A virtual
{ /*略*/ }

D.
virtual class B: public A
{ /*略*/ }

我的答案: A 正确答案: B

✗

0 分

44. (单选题)考虑到多重继承可能导致命名冲突问题, 常见的解决方案是:

A. 限定只能单继承

B. 虚基类

C. 限定多个基类中, 至多只能有一个基类有实例变量

D. 以上都包括

我的答案: D 正确答案: D

✓

0.9 分

45. (单选题)下面选项中最应该是工具类的是:

A.
class Telephone {
public:
 void callTo();
 void callBy();
private:
 /* 略 */
};

B.
class IComputer
{
public:
 static const int xx = 1;

作业详情

```
virtual void runApp( );
virtual void showVideo( );
};

C.
class Mobile:public Telephone,public IComputer
{
public:
    void callTo( );
    void callBy( );
    virtual void runApp( );
    virtual void showVideo( );
};

D.
class Package
{
public:
    static void PackBinary( );
    static void PackText( );
    static void PackPicture( );
    static int  getVer( );
private:
    static int  version;
};
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

我的答案: D 正确答案: D  0.9 分

46. (单选题)已定义了类A、类B和类C，其中类C采用多重继承的方式，从类A和类B派生，如下：

```
class A {
public:
    void fA( );
};

class B {
public:
    void fB( );
};

class C:private A, public B {
public:
    void fC( );
};
```

那么若改类C为单继承，那么最合适的是：

```
A.
class C: public A
{
public:
    void fB( );
    void fC( );
private:
    B  mb;
};

B.
class C: public B
{
public:
    void fA( );
    void fC( );
private:
    A  ma;
};

C.
class C: public A {
public:
```

作业详情

```
void fB( );
B mb;
};

D.
class C: public B
{
public:
    void fC( );
private:
    void fA( );
    A ma;
};
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

我的答案: A 正确答案: D  0 分

答案解析:

47. (单选题)多重继承会产生命名冲突，例如在如下的菱形结构中：

```
struct A {
    void f();
    int a;
};

struct B:public A {
    void g();
    int b;
};

struct C: public A {
    void h();
    int c;
};

struct D: public B, public C {
    void k();
    int d;
};
```

采用虚基类会改变D类对象的存储结构，但对于D中成员函数 f 的命名冲突并没有过多的处理。

C++语言这样做一定是有道理的，那么合理的说法是：

- A. D中的两个f函数，是以重载的形式出现的，不用虚基类也没问题，因此解决函数冲突不是虚基类的主要目的
- B. 虚基类本质上是解决地址冲突的问题。实际上虚基类在改变了对象存储结构的同时，也采用类似的方法，改变了函数的存储结构
- C. D中两个a，同名但地址不同；但D中两个f函数，同名而且实际地址也相同的。因此解决实例变量的地址冲突问题成为虚基类的主要目的
- D. D中两个a分别来自类B和类C，其类型、大小可能会不同；但D中两个f函数的内部表示都是指针，大小固定。因此解决实例变量的地址冲突问题成为虚基类的主要目的

我的答案: C 正确答案: C  0.9 分

答案解析:

二. 多选题 (共15题，14.7分)

48. (多选题)在C++程序设计时，应尽可能降低文件的编译期依赖性。这里所说的依赖性，是指：

- A. 一个头文件依赖另一个头文件
- B. 一个cpp文件依赖另一个cpp文件
- C. 一个cpp文件依赖另一个头文件

我的答案: ABCD

正确答案: AC

✖

0分

49. (多选题)关于降低文件编译期依赖性的益处，正确的是：
- A. 可以加快程序的编译速度
 - B. 减少对使用者的不必要语法限制，如去掉cpp文件中include语句的前后顺序要求
 - C. 提高复用性，利于后期的修改、升级、移植等
 - D. 降低由于程序缺陷而引起的缺陷扩散范围和影响

我的答案: ABCD

正确答案: ABCD

✔

0.9分

答案解析：

50. (多选题)

已知a.h文件中定义了类A，且b.h文件如下：

```
//b.h文件
#pragma once
#include "a.h"
class B {
public:
    void f() { mA.func(); }
    void g() { }
private:
    A mA;
};
```

可见b.h文件中include了a.h文件。若希望去掉b.h文件中的include语句，可以采用的合理做法是：

- A. 将#include "a.h"换成class A;，以外联方式实现B::f函数。
- B. 将#include "a.h"换成class A;，以外联方式实现B::f函数和B::g函数。
- C. 将#include "a.h"换成class A; 同时重新定义类B如下

```
class B {
public:
    B(): mpA(new A) { }
    void f() { mpA->func(); }
    void g() { }
    ~B() { delete mpA; }
private:
    A * mpA;
};
```
- D. 将#include "a.h"换成class A; 同时重新定义类B，并采用外联实现类B的构造、析构和f 函数，如下

```
class B {
public:
    B();
    void f();
    void g() { }
    ~B();
private:
    A * mpA;
};

//外联实现
B::B(): mpA(new A) { }
void B::f() { mpA->func(); }
B::~B() { delete mpA; }
```
- E.

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

作业详情

```
public:
    B();
    void f();
    void g();
    ~B();
private:
    A * mpA;
};

//外联实现
B::B(): mpA(new A) { }
void B::f() { mpA->func(); }
void B::g() { }
B::~B() { delete mpA; }
```

F.

将#include "a.h"换成class A; 同时重新定义类B, 并采用外联实现类B的所有成员函数, 如下

```
class B {
public:
    B(A * p);
    void f();
    void g();
    ~B();
private:
    A * mpA;
};

//外联实现
B::B(A * p): mpA(p) { }
void B::f() { mA.func(); }
void B::g() { }
B::~B() { delete mpA; }
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

我的答案: CDEF 正确答案: DE  0 分

答案解析:

51. (多选题)//a.h文件如下

```
#pragma once
class A {
public:
    void fA() {}
};
```

//b.h文件如下

```
#pragma once
class B {
public:
    void fB(A & a) { a.fA(); }
};
```

//main.cpp文件如下

```
#include "a.h"
#include "b.h"
int main() {
    A a;
    B b;
    b.fB(a);
}
```

上边给出的程序是能够编译并正确执行的。但程序中仍存在不妥的地方, 正确说法是:

- A. 由于fB采用内联实现, 应在b.h中添加class A;
- B. 由于fB采用内联实现, 应在b.h中添加#include "a.h"

D. 若fB改用外联实现，应在b.h中添加#include "a.h";

我的答案: ABCD

正确答案: BC

✖

0分

答案解析:

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

52. (多选题)下面给出定义对象型数据成员或指针型数据成员的形式,

//形式1

```
class A{
private:
    B mB;
};
```

//形式2

```
class A{
private:
    B * mpB;
};
```

那么下面说法正确的是:

- A. 形式1相比形式2, 编译期依赖性更强
- B. 形式1体现类A和类B间的逻辑关系较单一, 而形式2可以表示类A和类B间的多种逻辑关系
- C. 形式2更灵活, 但对程序员的设计和编码能力要求更高
- D. 形式2通常需要考虑类B的构造、析构、拷贝、赋值等行为的定义和实现

我的答案: ABCD

正确答案: ABCD

✔

1分

53. (多选题)

```
class A {
    /*略*/
private:
    B b;
};
```

那么类A和类B间的逻辑关系可以是:

- A. 一般关联关系
- B. 聚合关系
- C. 组合关系
- D. 依赖关系

我的答案: ABC

正确答案: AC

✖

0分

答案解析:

54. (多选题)

```
class A {
    /*略*/
private:
    B * pb;
};
```

那么类A和类B间的逻辑关系可以是:

- A. 一般关联关系
- B. 聚合关系

D. 依赖关系

我的答案: ABC 正确答案: ABC  1分

55. (多选题)

```
//b.h文件
#pragma once
#include <iostream>
using namespace std;
class B {

};

//a.h文件
#pragma once
#include "b.h"
class A {
public:
    int f() const { return 1; }
private:
    B * mpb;
};

//main.cpp文件
#include <iostream>
using namespace std;
#include "a.h"
int main() {
    A a;
    cout<<a.f();
}
为降低上边三个文件的编译期依赖性，必须且合理的处理是：

A. 去掉b.h文件中的include和using语句



B. 将a.h文件中的include "b.h"语句改成class B;



C. 将a.h文件中A::f函数，改为外联实现



D. 在main.cpp文件中，添加#include "b.h"


```

我的答案: AB 正确答案: AB  1分

答案解析：

56. (多选题)若定义Dorm类和Student类，分别存为dorm.h和student.h，同时希望Dorm组合Student，Student知道其所在的Dorm，现定义如下：

```
//dorm.h
#pragma once
#include "student.h"
class Dorm {
private:
    Student mStudents[30];
};

//student.h
#pragma once
#include "dorm.h"
class Student {
private:
    Dorm mDorm;
};
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

作业详情

```
#include "student.h"
int main() {
    /* 略 */
}
```

上边代码编译不会通过的。若希望保持Dorm类的原有类定义不改变，如何合理地修改各文件并使得程序能通过编译？

- A. 将dorm.h中的#include "student.h"，改为class Student;
- B. 将student.h中的#include "dorm.h"，改为class Dorm;
- C. 将Student类中定义的数据成员mDorm修改为 Dorm * mpDorm;
- D. 对main.cpp文件中的两条include语句，改其中一个为前置声明

我的答案: BC

正确答案: BC

✓

1分

57. (多选题)已有main.cpp文件内容如下:

```
#include <iostream>
using namespace std;
#include "b.h" //位置1
class A {
public:
    A():mpB(nullptr) {}
    void setup() { mpB = new B; }
    B* getB() const { return mpB; }
private:
    B * mpB;
};
//位置2
int main() {
    A a;
    a.setup();
    delete a.getB();
}
```

那么，上边关于上边程序的说法正确的是:

- A. 将位置1的#include语句移动到位置2，同时将位置1改写成class B, 是可以编译通过的。
- B. A和B的逻辑关系，可以是A组合B
- C. 创建B类对象和释放B类对象的过程分布在程序不同地方，这不是一个好的编程习惯
- D. 定义类A时，还应该考虑类A的拷贝构造以及赋值行为的定义和实现

我的答案: CD

正确答案: CD

✓

1分

58. (多选题)在程序中体现中国和长春之间的关系，可能的有:

- A. Country类聚合City类，Country类实例化出中国对象，City类实例化出长春对象
- B. Country类组合City类，Country类实例化出中国对象，City类实例化出长春对象
- C. Country类关联City类，Country类实例化出中国对象，City类实例化出长春对象
- D. Country类的成员函数以City类为参数，Country类实例化出中国对象，City类实例化出长春对象

我的答案: ABCD

正确答案: ABCD

✓

1分

59. (多选题)

针对下面给出的代码:

```
#include <iostream>
using namespace std;
```

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

```
A (int n=1):mVal(n)          { cout<<1; }
A (const A& rhs):mVal(rhs.mVal) { cout<<2;}
A& operator=(const A& )      { cout<<3; return *this;}
~A()                          { }
int mVal;
};

class B:public A {
public:
    B()          { cout<<4; }
    B(const B&)  { cout<<5; }          //1
    B& operator=(const B& ) { cout<<6; return *this;} //2
};

int main() {
    B b1;
    B b2(b1);
    b1 = b2;
}
```

现在将代码中的//1和//2两行注释掉，那么输出结果会不一样。
分析执行结果以及原因，回答下面哪些说法是正确的？

A. 构造子类时，无论使用普通构造函数还是拷贝构造函数，都会先构造基类

B. 子类缺省的拷贝构造函数会隐式调用基类的拷贝构造函数

C. 若自定义子类的拷贝构造函数，那么它也会隐式调用基类的拷贝构造函数

D. 子类缺省的赋值函数会隐式调用基类的赋值函数

E. 若自定义了子类的赋值函数，那么基类数据成员如何赋值完全由子类赋值函数负责

我的答案: ABCE

正确答案: ABDE

✖

0分

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

60. (多选题)

在定义一个类Base时，可以指定其成员的访问控制为public或protected或private，假如定义如下：

```
class Base {
public:      void f();      //1
protected: void g();      //2
protected: int mProtVar;  //3
private:   void k();       //4
private:   int mPrivVar;   //5
};
```

那么关于什么时候将成员指定为protected访问控制的说法，其中合理的说法是：

A. 若Base只能以protected继承方式派生其它类，那么应将//4、//5都应改成protected的

B. 若只允许以public继承方式派生其它类，那么//2、//3应该改成public或private的

C. 若只允许以public继承方式派生其它类，那么//2中的protected意味着Base的所有后裔类中都可以直接访问g()

D. 若只允许以public继承方式派生其它类，那么//3应改成private的，同时提供protected的getter/setter函数

E. 无论哪种继承方式，//4中的private会导致后裔类都不能访问k()，也就失去了定义和实现k()的意义，至少应将//4改为protected的

我的答案: BCD

正确答案: CD

✖

0分

61. (多选题)

已知Base类定义如下：

```
class Base {
public:
    Base(int n):mBaseVal(n) {}
    Base(const Base& rhs) =default;
    Base& operator=(const Base& rhs)=default;
    void f() { /*略*/ }
protected:
    // 1
```

https://mooc1.chaoxing.com/mooc-ans/mooc2/work/view?courseId=201642417&classId=93222363&cpi=352876303&workId=35022770&answ...

28/37

作业详情

```
private:           // 2
    int mBaseVal;
};

且Derived的部分定义如下:
class Derived:private Base {
public:
    /* 其它成员函数 */           // 3
    void g() { /*略*/ }
    int value() const { return mDerivedVal + baseValue(); }
    void setValue(int n) { mDerivedVal = n; }
private:           //4
    int mDerivedVal;
};
```

那么下列哪些是合理的(不必考虑虚函数的情况下):

- A. //3中必须有自定义的构造函数
- B. //3中必须有自定义的析构函数
- C. //3中必须有自定义的拷贝构造函数
- D. //3中必须有自定义的赋值函数
- E. //1中的protected应改成private
- F. //2中的private改成protected在语法上是允许的，但从设计角度来说，这样做不好。
- G. 考虑到类Derived未来可能派生新的类，//4中的private改成protected更合适。

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

我的答案: ABCDF 正确答案: AF



0 分

答案解析:

62. (多选题)

已给出类Base，类Parent，以及类Child的定义如下:

```
class Base {
public:
    Base(int n):mVal0(n) {}
    void pubF() { cout<<"pubF()"<<endl;}
protected:
    void protF() { cout<<"protF()"<<endl;}
private:
    void privF() { cout<<"privF()"<<endl;}
private:
    int mVal0;
};

class Parent {
public:
    Parent(int n1):mVal1(n1){}
    void pubG() { cout<<"pubG()"<<endl;}
protected:
    void protG() { cout<<"protG()"<<endl;}
private:
    void privG() { cout<<"privG()"<<endl;}
private:
    int mVal1;
};

class Child: public Parent, private Base {
public:
    Child(int n1,int n2,int n3):Parent(n1),Base(n2),mVal2(n3) {}
    void pubH() {cout<<"pubH()"<<endl;}
protected:
    void prothH() {cout<<"prothH()"<<endl;}
private:
    void privH() { cout<<"privH()"<<endl;}
    int mVal2;
};
```

作业详情

现欲将Child类的定义从多重继承改为单继承，并保持行为语义不变。部分定义如下：

```
class Child:public Parent {
public:
    Child(int n1,int n2,int n3):Parent(n1),pBase(new Base(n2)),mVal(n3) {}
    /*其它，待补充*/    //1
private:
    Base * pBase;
    /* 待补充 */        //2
};
```

那么在不考虑虚函数的情况下，下列说法正确的是：

- A. //1中需要自定义public的析构函数
- B. //1中需要自定义的public的拷贝构造函数
- C. //1中需要自定义public的赋值函数
- D. //1中需要自定义public的pubF函数
- E. //1中需要自定义private的pubF函数
- F. //1中需要自定义protected的protF函数
- G. //1中需要自定义private的protF函数
- H. //2中需要自定义private的int mVal0变量
- I. //2中需要自定义private的int mVal1变量

我的答案: ABC

正确答案: ABCEG



0.5 分

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

三. 判断题 (共43题, 43分)

63. (判断题)在真正应用开发中，一般定义一个类并实例化该类的对象，就足以完成应用的功能了。

- A. 对
- B. 错

我的答案: 错

正确答案: 错



1 分

64. (判断题)弱关联是最常见的类间联系，也是最弱的类间联系。

- A. 对
- B. 错

我的答案: 错

正确答案: 错



1 分

65. (判断题)自关联强调的是对象间的关系。

- A. 对
- B. 错

我的答案: 错

正确答案: 错



1 分

66. (判断题)具有组合关系的两个类，他们的对象之间一定具有着“同生共死”的关系。

- A. 对

作业详情

我的答案: 错 正确答案: 错

✓

1分

67. (判断题)尽可能将行为放在主动的多变的类中，方便以后的扩展。

A. 对

B. 错

我的答案: 对 正确答案: 对

✓

1分

68. (判断题)对已有类的复用，应该根据实际需要进行适当修改。

A. 对

B. 错

我的答案: 错 正确答案: 错

✓

1分

69. (判断题)黑盒复用是一种功能复用，它真正关心的是被复用类有哪些数据成员，成员函数是如何实现的。

A. 对

B. 错

我的答案: 错 正确答案: 错

✓

1分

70. (判断题)黑盒复用中，被复用类应具有良好的设计，且需要完整的源实现代码。

A. 对

B. 错

我的答案: 错 正确答案: 错

✓

1分

71. (判断题)继承方式默认为private继承。

A. 对

B. 错

我的答案: 对 正确答案: 对

✓

1分

72. (判断题)一般地，基类可称为父类，派生类可称为子类。

A. 对

B. 错

我的答案: 错 正确答案: 错

✓

1分

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

作业详情

B. 错

我的答案: 错

正确答案: 错

✓

1分

74. (判断题)类 B 继承类 A，那么sizeof(B)的值大于sizeof(A)的值。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1分

75. (判断题)基类中private的成员在派生类中都是不可访问的，因此派生类不需要含有这些不可访问的基类成员。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1分

76. (判断题)基类中private的成员在派生类中是无法访问的。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1分

答案解析：基类中private的成员在派生类中不允许直接访问，但可以通过基类公有成员函数间接访问。

77. (判断题)基类中protected的成员在public继承或protected继承下无论继承多少层都仍然是protected。

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1分

78. (判断题)已知类B以私有方式继承类A，那么类C无论以何种方式继承类B，在类C的成员函数函数体内，都不能直接访问类A的成员。

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1分

79. (判断题)类A中有protected的数据成员int n，类B以公有方式继承类A，那么类B的成员函数：void F(A & aA) { aA.n = 10; }是错误的。

- A. 对
- B. 错

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

作业详情

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

80. (判断题)可以在派生类构造函数的成员初始化列表直接初始化基类的数据成员。

- A. 对
- B. 错

我的答案: 错 正确答案: 错

✓

1分

81. (判断题)自定义派生类构造函数的初始化列表中如果不显式调用基类构造函数，则自动调用基类的拷贝构造函数。

- A. 对
- B. 错

我的答案: 错 正确答案: 错

✓

1分

答案解析:

82. (判断题)在多继承情况下，派生类构造函数中基类构造函数的执行顺序取决于定义派生类时所指定的各基类的顺序。

- A. 对
- B. 错

我的答案: 对 正确答案: 对

✓

1分

83. (判断题)自定义派生类拷贝构造函数的初始化列表中如果不显式调用基类拷贝构造函数，则自动调用基类的拷贝构造函数。

- A. 对
- B. 错

我的答案: 错 正确答案: 错

✓

1分

答案解析: 自定义派生类拷贝构造函数的初始化列表中如果不显式调用基类拷贝构造函数，则自动调用基类的缺省构造器。

缺省构造器是指：编译器提供的缺省构造函数或自定义无参构造函数或自定义所有参数都有默认实参的带参构造函数。

84. (判断题)面向对象在类设计时优先选择“一堆继承树组成的低矮的灌木丛”，而不是“一颗非常高的孤立的继承树”。

- A. 对
- B. 错

我的答案: 对 正确答案: 对

✓

1分

答案解析: 优先选择组合而不是继承，继承的层次最好保持在3-5层。

85. (判断题)从基类向派生类进行类型转换一般称为向上类型转换。

- A. 对

作业详情

我的答案: 错 正确答案: 错  1分

86. (判断题)protected和private继承下的向上类型转换在逻辑上是无实际意义的。

- A. 对
- B. 错

我的答案: 对 正确答案: 对  1分

87. (判断题)逻辑上，父类类型是子类类型的泛化或一般化。

- A. 对
- B. 错

我的答案: 对 正确答案: 对  1分

88. (判断题)将子类对象转换成父类对象是安全的，但原来的子类对象不存在了。

- A. 对
- B. 错

我的答案: 错 正确答案: 错  1分

89. (判断题)应尽可能使用指针或引用型向上类型转换。

- A. 对
- B. 错

我的答案: 对 正确答案: 对  1分

90. (判断题)

已知类A和类B，存在A a;则语句B* p1 = (B*)&a;和B* p = static_cast <B* > (&a);是等价的。

- A. 对
- B. 错

我的答案: 错 正确答案: 错  1分

答案解析:

91. (判断题)

C++静态类型转换时建议使用static_cast<T>(exp)，尽量不使用(T)exp。

- A. 对
- B. 错

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

答案解析:

92. (判断题)Volatile关键字修饰的变量具有易变性和不可优化性。

A. 对

B. 错

我的答案: 对

正确答案: 对

✓

1分

93. (判断题)常成员函数无法修改所属类中的非静态数据成员。

A. 对

B. 错

我的答案: 错

正确答案: 错

✓

1分

存在语句:

int m=10;

double d1=static_cast<double>(m);

double d2=reinterpret_cast<double &>(m);

则d1与d2是相等的。

94. (判断题)

A. 对

B. 错

我的答案: 错

正确答案: 错

✓

1分

答案解析:

95. (判断题)向下类型转换一定是不安全的。

A. 对

B. 错

我的答案: 错

正确答案: 错

✓

1分

96. (判断题)通常情况下，静态类型转换发生在程序编译期间，动态类型转换发生在程序运行期间。

A. 对

B. 错

我的答案: 对

正确答案: 对

✓

1分

一. 单选题 (42.3分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38
41	42	43

https://mooc1.chaoxing.com/mooc-ans/mooc2/work/view?courseId=201642417&classId=93222363&cpi=352876303&workId=35022770&answ...

35/37

作业详情

一. 单选题 (42.3分)

- 123
- 678
- 111213
- 161718
- 212223
- 262728
- 313233
- 363738
- 414243

A. 对

B. 错

我的答案: 对 正确答案: 对

✓

1分

98. (判断题)多重继承下，派生类的构造顺序严格按照初始化列表的顺序依次构造，而析构顺序与构造顺序相反。

A. 对

B. 错

我的答案: 错 正确答案: 错

✓

1分

99. (判断题)无论是单继承还是多重继承，基类中的数据成员都会被派生类继承，并存在于派生类对象的存储空间内。

A. 对

B. 错

我的答案: 错 正确答案: 错

✓

1分

100. (判断题)多重继承下，定义派生类应采用虚基类的继承方式。

A. 对

B. 错

我的答案: 错 正确答案: 错

✓

1分

101. (判断题)一般情况下，在派生类对象中，同名的虚基类只产生一个虚基类子对象，而非虚基类产生各自的子对象。

A. 对

B. 错

我的答案: 对 正确答案: 对

✓

1分

102. (判断题)接口类可以实例化。

A. 对

B. 错

我的答案: 错 正确答案: 错

✓

1分

103. (判断题)接口继承要求基类均为接口类。

A. 对

B. 错

我的答案: 错 正确答案: 错

✓

1分

作业详情

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1分

105. (判断题)对于public继承方式下的多重继承，与单继承一样，用基类的指针或引用只能访问基类中定义（或继承）的成员，不能访问派生类中新引入的成员。

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1分

一. 单选题 (42.3分)

- 1

2

3
- 6

7

8
- 11

12

13
- 16

17

18
- 21

22

23
- 26

27

28
- 31

32

33
- 36

37

38
- 41

42

43